

Table of Contents

FOIL Algorithm and Beam Search Algorithm Implementation on a Given Dataset

Student Information

Name: _____

ID: _____

Course: Machine Learning

Submission Date: _____

Bangla Explanation Section

1. Bangla Topic Description

FOIL Algorithm কী?

FOIL এর পূর্ণরূপ First Order Inductive Learner। এটি একটি rule learning algorithm। সহজভাবে বললে, FOIL dataset-এর positive এবং negative examples দেখে logical IF-THEN rule শেখে।

Example:

IF outlook = overcast THEN play_tennis = yes

এর অর্থ হলো, যদি weather outlook overcast হয়, তাহলে tennis খেলা হবে।

FOIL সাধারণত logic programming style-এ কাজ করে। এটি এমন rule খুঁজে বের করার চেষ্টা করে যেগুলো positive example cover করে, কিন্তু negative example cover করে না।

Beam Search Algorithm কী?

Beam Search হলো একটি heuristic search algorithm। এটি সব possible solution একসাথে ধরে রাখে না। প্রতিটি level বা step-এ অনেক candidate তৈরি হয়, কিন্তু beam width অনুযায়ী শুধু best few candidates রাখা হয়।

Example:

Beam width = 3

এর মানে হলো, প্রতিটি step-এ শুধু সেরা ৩টি candidate রাখা হবে। বাকি candidate বাদ দেওয়া হবে।

Beam Search তখন কাজে লাগে যখন possible solution অনেক বেশি হয় এবং সব solution search করা time-consuming হয়।

Prolog কী?

Prolog হলো একটি logic programming language। এখানে program লেখা হয় facts, rules, এবং queries দিয়ে।

Fact example:

```
outlook(d1, sunny).
```

Meaning:

d1 দিনের outlook sunny।

Rule example:

```
foil_rule(Day) :-  
    outlook(Day, overcast).
```

Meaning:

যে Day-এর outlook overcast, সেই Day-এর জন্য foil_rule true হবে।

কেন FOIL algorithm-এর সাথে Prolog সম্পর্কিত?

FOIL logical rule শেখে। Prolog logical fact এবং rule ব্যবহার করে reasoning করে। তাই FOIL থেকে শেখা rule খুব সহজে Prolog-এ লেখা যায়।

FOIL rule:

```
IF outlook = rainy AND wind = weak THEN play_tennis = yes
```

Prolog version:

```
foil_rule(Day) :-  
    outlook(Day, rainy),  
    wind(Day, weak).
```

এই কারণে FOIL এবং Prolog একই logic programming ধারণার সাথে সম্পর্কিত।

কেন এই assignment-এ dataset দরকার?

Dataset ছাড়া FOIL rule শিখতে পারবে না এবং Beam Search candidate rule evaluate করতে পারবে না। Dataset থেকে algorithm বুঝতে পারে কোন input condition-এর জন্য output yes বা no হয়।

এই assignment-এ dataset থেকে পাওয়া যায়:

- Input attributes: outlook, temperature, humidity, wind
- Target/output column: play_tennis

- Positive examples: যেসব row-তে play_tennis = yes
- Negative examples: যেসব row-তে play_tennis = no

এই assignment-এর main purpose কী?

এই assignment-এর main purpose হলো:

- FOIL algorithm কীভাবে logical rule শেখে তা implement করা
- Beam Search কীভাবে best candidate rule খুঁজে বের করে তা implement করা
- একই dataset ব্যবহার করে দুই algorithm-এর কাজ বোঝানো
- Python code, Prolog code, output এবং documentation তৈরি করা
- Prolog-এর facts, rules, queries beginner-friendly ভাবে ব্যাখ্যা করা

2. Assignment Workflow in Bangla

Step 1: Dataset selection

এই project-এ Play Tennis dataset ব্যবহার করা হয়েছে। এটি weather-based classification dataset।

Dataset attributes:

Attribute	Meaning	Example Values
outlook	আবহাওয়ার অবস্থা	sunny, overcast, rainy
temperature	তাপমাত্রা	hot, mild, cool
humidity	আর্দ্রতা	high, normal
wind	বাতাসের অবস্থা	weak, strong
play_tennis	target/output class	yes, no

Target/output column:

play_tennis

অর্থাৎ, given weather condition অনুযায়ী tennis খেলা হবে কি না সেটাই output।

Step 2: Input preparation

Python program dataset নেয় dataset/data.csv file থেকে। CSV file-এর প্রতিটি row একটি example।

Positive examples:

যেসব row-তে play_tennis = yes

এই project-এর positive examples:

d3, d4, d5, d7, d9, d10, d11, d12, d13

Negative examples:

যেসব row-তে play_tennis = no

এই project-এর negative examples:

d1, d2, d6, d8, d14

Prolog facts তৈরি করা হয়েছে dataset/facts.pl file-এ। প্রতিটি dataset row Prolog fact আকারে লেখা হয়েছে।

Example:

```
day(d9).  
outlook(d9, sunny).  
temperature(d9, cool).  
humidity(d9, normal).  
wind(d9, weak).  
play_tennis(d9, yes).
```

এই facts ব্যবহার করে Prolog query চালানো যায়।

Step 3: FOIL implementation

FOIL positive এবং negative examples থেকে rule শেখে।

FOIL-এর rule শেখার process:

1. প্রথমে positive এবং negative examples আলাদা করা হয়।
2. Candidate condition তৈরি করা হয়, যেমন outlook = overcast।
3. প্রতিটি candidate rule কতগুলো positive এবং negative example cover করে তা check করা হয়।
4. যে rule বেশি positive cover করে এবং negative কম বা 0 cover করে, সেটি ভালো rule হিসেবে ধরা হয়।
5. Selected rule দিয়ে যেসব positive example cover হয়ে যায়, সেগুলো remove করা হয়।
6. সব positive example cover না হওয়া পর্যন্ত process repeat হয়।

Python FOIL code কী করে:

- data.csv read করে
- positive এবং negative examples আলাদা করে
- possible candidate rules তৈরি করে
- FOIL gain/score ব্যবহার করে best rule select করে
- final learned rules print করে
- প্রতিটি dataset row classify করে
- accuracy print করে

এই project-এ learned FOIL rules:

Rule 1: IF outlook = overcast THEN play_tennis = yes

Rule 2: IF outlook = rainy AND wind = weak THEN play_tennis = yes

Rule 3: IF outlook = sunny AND humidity = normal THEN play_tennis = yes

Default: otherwise play_tennis = no

Prolog FOIL/rule code কী করে:

- facts.pl file load করে
- learned FOIL rules Prolog rule হিসেবে define করে
- classify(Day, Result) query দিয়ে prediction দেয়
- test_all দিয়ে সব row test করে
- accuracy দিয়ে final accuracy দেখায়

Step 4: Beam Search implementation

Beam Search candidate states বা candidate rules search করে।

Initial state:

IF TRUE THEN play_tennis = yes

এই rule খুব general, কারণ এটি সব row-কে yes predict করে।

Beam width:

3

এর অর্থ হলো প্রতিটি level-এ only best 3 states রাখা হবে।

Candidate states কী?

Candidate states হলো possible rules। Example:

IF outlook = overcast THEN play_tennis = yes

IF humidity = normal THEN play_tennis = yes

IF outlook = rainy AND wind = weak THEN play_tennis = yes

Scoring function কী?

Beam Search প্রতিটি candidate rule score করে। এই project-এ score করা হয়েছে precision, recall এবং purity bonus দিয়ে।

$\text{score} = 0.70 * \text{precision} + 0.30 * \text{recall} + \text{purity bonus}$

Precision:

Rule যতগুলো example cover করেছে, তার মধ্যে কতগুলো সত্যিই positive।

Recall:

Total positive example-এর মধ্যে rule কতগুলো positive cover করেছে।

Best candidates কীভাবে select হয়?

প্রতিটি level-এ সব candidate rule score করা হয়। তারপর score অনুযায়ী sort করা হয় এবং top 3 rule beam হিসেবে রাখা হয়। এরপর next level-এ শুধু এই selected rules expand করা হয়।

Beam Search code কী output দেখায়:

- Initial state
- Beam width
- Candidate states
- প্রতিটি candidate-এর precision, recall, score
- প্রতিটি level-এ selected beam states
- Final best rule

Step 5: Output generation

FOIL Python output কী?

FOIL Python output দেখায়:

- Positive examples
- Negative examples
- প্রতিটি iteration-এর candidate rules
- Selected rule
- Final learned rules
- Classification result
- Accuracy

Prolog output কী?

Prolog output দেখায়:

- কোনো query true না false
- কোনো day-এর prediction yes/no
- সব row-এর predicted এবং actual result
- final accuracy

Beam Search output কী?

Beam Search output দেখায়:

- প্রতিটি level-এর candidate states
- Candidate score
- Beam width অনুযায়ী selected best states
- Final best rule

Output থেকে কী বোঝা যায়?

Output দেখে বোঝা যায় কোন rules dataset-এর positive examples ভালোভাবে explain করে। FOIL final rule set তৈরি করে, আর Beam Search search process দেখিয়ে best candidate rule খুঁজে বের করে।

3. Input and Output Section

Input

Dataset file:

dataset/data.csv

এই file Python FOIL এবং Beam Search code-এর input হিসেবে ব্যবহার করা হয়েছে।

Prolog facts file:

dataset/facts.pl

এই file Prolog code-এর input knowledge base হিসেবে কাজ করে।

Positive examples:

d3, d4, d5, d7, d9, d10, d11, d12, d13

Negative examples:

d1, d2, d6, d8, d14

Beam width:

3

Search states/candidate rules:

IF outlook = overcast THEN play_tennis = yes

IF humidity = normal THEN play_tennis = yes

IF wind = weak THEN play_tennis = yes

IF outlook = rainy AND wind = weak THEN play_tennis = yes

Output

Learned rules from FOIL:

IF outlook = overcast THEN play_tennis = yes

IF outlook = rainy AND wind = weak THEN play_tennis = yes

IF outlook = sunny AND humidity = normal THEN play_tennis = yes

OTHERWISE play_tennis = no

Classification result:

d9: predicted=yes, actual=yes

Prolog query result:

```
?- play_tennis(d9, yes).  
true.
```

Beam Search selected best rules/states:

```
IF outlook = overcast THEN play_tennis = yes  
IF humidity = normal THEN play_tennis = yes  
IF wind = weak THEN play_tennis = yes
```

Step-by-step search trace:

```
Level 1: candidate rules are generated and scored  
Level 1: top 3 rules are kept  
Level 2: selected rules are expanded  
Level 2: new candidate rules are scored  
Final: best rule is printed
```

Final best result:

```
Best rule found: IF outlook = overcast THEN play_tennis = yes
```

4. Example Input and Output

Example input dataset row:

```
Outlook = Sunny  
Temperature = Cool  
Humidity = Normal  
Wind = Weak
```

এই input row dataset-এ d9 হিসেবে আছে:

```
d9,sunny,cool,normal,weak,yes
```

Expected output:

```
PlayTennis = Yes
```

Why?

কারণ FOIL learned rule অনুযায়ী:

```
IF outlook = sunny AND humidity = normal THEN play_tennis = yes
```

Example Prolog query:

```
?- play_tennis(d9, yes).
```

Expected Prolog output:

true.

Classification query:

```
?- classify(d9, Result).
```

Expected output:

Result = yes.

Note:

এই project-এ day ID হিসেবে d9 ব্যবহার করা হয়েছে। যদি day9 লিখতে চান, তাহলে dataset এবং Prolog facts file-এ ID একইভাবে পরিবর্তন করতে হবে।

1. Task Explanation

This assignment implements two algorithms:

1. FOIL algorithm
2. Beam Search algorithm

Both algorithms are demonstrated on the same small dataset called the Play Tennis dataset. The dataset describes weather conditions and whether tennis was played on that day.

What is FOIL?

FOIL means First Order Inductive Learner. It is a rule learning algorithm. It learns logical rules from examples.

In simple words, FOIL tries to learn rules like this:

```
play_tennis(Day) :- outlook(Day, overcast).
```

The meaning is:

“Tennis is played on a day if the outlook of that day is overcast.”

FOIL uses:

- Positive examples: examples where the target answer is yes.
- Negative examples: examples where the target answer is no.
- Background facts: known information about each example.

FOIL starts with a very general rule and then adds conditions until the rule covers positive examples and avoids negative examples.

What is Beam Search?

Beam Search is a heuristic search algorithm. It explores possible solutions level by level, but it does not keep all solutions. At every level, it keeps only the best few candidates.

The number of candidates kept is called the beam width.

Example:

- Beam width = 3
- At each level, keep only the best 3 candidate rules
- Remove the weaker candidates

Beam Search is useful when the search space is large and checking every possible solution would take too much time.

Why is Prolog related to FOIL?

Prolog is a logic programming language. It represents knowledge using facts and rules.

FOIL also learns logical rules. Therefore, Prolog is a natural language for showing FOIL-style rules.

Example Prolog fact:

```
outlook(d1, sunny).
```

This means:

“On day d1, the outlook is sunny.”

Example Prolog rule:

```
foil_rule(Day) :-  
    outlook(Day, rainy),  
    wind(Day, weak).
```

This means:

“Tennis is played on a day if the outlook is rainy and the wind is weak.”

How both algorithms are applied to the dataset

FOIL is used to learn classification rules for:

```
play_tennis = yes
```

Beam Search is used to search through possible IF-THEN rules and keep the best candidate rules according to a scoring function.

2. Dataset

The dataset used in this project is the Play Tennis dataset.

Dataset Table

ID	Outlook	Temperature	Humidity	Wind	Play Tennis
d1	sunny	hot	high	weak	no
d2	sunny	hot	high	strong	no
d3	overcast	hot	high	weak	yes
d4	rainy	mild	high	weak	yes
d5	rainy	cool	normal	weak	yes
d6	rainy	cool	normal	strong	no
d7	overcast	cool	normal	strong	yes
d8	sunny	mild	high	weak	no
d9	sunny	cool	normal	weak	yes
d10	rainy	mild	normal	weak	yes
d11	sunny	mild	normal	strong	yes
d12	overcast	mild	high	strong	yes
d13	overcast	hot	normal	weak	yes
d14	rainy	mild	high	strong	no

CSV Dataset File

File:

dataset/data.csv

Content:

```
id,outlook,temperature,humidity,wind,play_tennis
d1,sunny,hot,high,weak,no
d2,sunny,hot,high,strong,no
d3,overcast,hot,high,weak,yes
d4,rainy,mild,high,weak,yes
d5,rainy,cool,normal,weak,yes
d6,rainy,cool,normal,strong,no
d7,overcast,cool,normal,strong,yes
d8,sunny,mild,high,weak,no
d9,sunny,cool,normal,weak,yes
d10,rainy,mild,normal,weak,yes
d11,sunny,mild,normal,strong,yes
d12,overcast,mild,high,strong,yes
d13,overcast,hot,normal,weak,yes
d14,rainy,mild,high,strong,no
```

Prolog Dataset File

File:

```
dataset/facts.pl
```

The Prolog version stores the same dataset as facts such as:

```
day(d1).  
outlook(d1, sunny).  
temperature(d1, hot).  
humidity(d1, high).  
wind(d1, weak).  
play_tennis(d1, no).
```

3. FOIL Algorithm Implementation

FOIL Step-by-Step

FOIL learns rules using this idea:

1. Separate the examples into positive and negative examples.
2. Start with an empty rule.
3. Add a condition such as outlook = overcast.
4. Check how many positive and negative examples the rule covers.
5. Keep adding conditions until the rule covers no negative examples.
6. Save the rule.
7. Remove the positive examples already covered by that rule.
8. Repeat until all positive examples are covered.

Positive Examples

Positive examples are rows where play_tennis = yes.

```
d3, d4, d5, d7, d9, d10, d11, d12, d13
```

Negative Examples

Negative examples are rows where play_tennis = no.

```
d1, d2, d6, d8, d14
```

Background Predicates/Facts

The background facts describe each day.

Examples:

```
outlook(d9, sunny).  
temperature(d9, cool).
```

```
humidity(d9, normal).  
wind(d9, weak).
```

These facts allow Prolog and FOIL-style rules to reason about the data.

FOIL Rules Used in Prolog

The learned rules are:

```
foil_rule(Day) :-  
    outlook(Day, overcast).
```

```
foil_rule(Day) :-  
    outlook(Day, rainy),  
    wind(Day, weak).
```

```
foil_rule(Day) :-  
    outlook(Day, sunny),  
    humidity(Day, normal).
```

In English:

1. If outlook is overcast, play tennis.
2. If outlook is rainy and wind is weak, play tennis.
3. If outlook is sunny and humidity is normal, play tennis.
4. Otherwise, do not play tennis.

How to Run the Python FOIL Program

Open a terminal in the project folder:

```
foil-beam-assignment
```

Run:

```
python foil/foil_python.py
```

To save output to a file:

```
python foil/foil_python.py > outputs/foil_python_output.txt
```

Expected Python FOIL Output Summary

The output shows:

- Positive examples
- Negative examples
- Candidate rules in each iteration
- Selected rule in each iteration

- Final learned rules
- Classification output
- Accuracy

Expected final rules:

Rule 1: IF outlook = overcast THEN play_tennis = yes

Rule 2: IF outlook = rainy AND wind = weak THEN play_tennis = yes

Rule 3: IF outlook = sunny AND humidity = normal THEN play_tennis = yes

Default rule: otherwise play_tennis = no

Accuracy: 14/14 = 100.00%

4. Beam Search Algorithm Implementation

Beam Search Step-by-Step

In this project, Beam Search searches for a good rule for predicting:

play_tennis = yes

Initial state:

IF TRUE THEN play_tennis = yes

This rule is too general because it predicts yes for every row.

The algorithm then creates candidate states such as:

IF outlook = overcast THEN play_tennis = yes

IF humidity = normal THEN play_tennis = yes

IF wind = weak THEN play_tennis = yes

Each candidate is scored using:

$\text{score} = 0.70 * \text{precision} + 0.30 * \text{recall} + \text{purity bonus}$

Where:

- Precision means how many covered examples are actually positive.
- Recall means how many total positive examples are covered.
- Purity bonus is added when the rule covers no negative examples.

Beam width:

3

This means the algorithm keeps only the best 3 candidates at each level.

How to Run Beam Search

Open a terminal in the project folder:

foil-beam-assignment

Run:

```
python beam_search/beam_search.py
```

To save output to a file:

```
python beam_search/beam_search.py > outputs/beam_search_output.txt
```

Beam Search Output Explanation

The output shows:

- Initial state
- Beam width
- Maximum depth
- Candidate states at each level
- Score for each candidate
- Selected beam states
- Final best rule

Example final result:

Best rule found: IF outlook = overcast THEN play_tennis = yes

This is a strong rule because all overcast examples in the dataset are positive examples.

5. Prolog Beginner Section

What is Prolog?

Prolog is a programming language based on logic. Instead of writing many step-by-step instructions, we write facts and rules. Then we ask questions called queries.

What are facts?

A fact is something that is directly true.

Example:

```
outlook(d1, sunny).
```

Meaning:

“The outlook on day d1 is sunny.”

Important Prolog syntax:

- Names like outlook, sunny, and d1 are atoms.
- Every fact ends with a period.

What are rules?

A rule defines something using conditions.

Example:

```
foil_rule(Day) :-  
    outlook(Day, overcast).
```

Meaning:

“The FOIL rule is true for Day if the outlook of Day is overcast.”

The symbol :- means “if”.

What are queries?

A query is a question asked to Prolog.

Example:

```
?- outlook(d1, sunny).
```

Prolog answers:

```
true.
```

Another query:

```
?- classify(d9, Result).
```

Prolog answers:

```
Result = yes.
```

How to Install SWI-Prolog

1. Go to <https://www.swi-prolog.org/>
2. Download SWI-Prolog for your operating system.
3. Install it using the default options.
4. Open SWI-Prolog from the Start Menu or terminal.

How to Save a .pl File

Prolog files use the .pl extension.

Example:

```
foil_prolog.pl
```

The files in this project are already saved in:

```
dataset/facts.pl  
foil/foil_prolog.pl
```

How to Load and Run the Prolog File

Option 1: Open SWI-Prolog inside the foil folder and type:

```
?- [foil_prolog].
```

Option 2: Open SWI-Prolog inside the main project folder and type:

```
?- ['foil/foil_prolog.pl'].
```

Sample Prolog Queries

```
?- foil_rule(d3).
```

Expected:

```
true.
```

Query:

```
?- foil_rule(d1).
```

Expected:

```
false.
```

Query:

```
?- classify(d9, Result).
```

Expected:

```
Result = yes.
```

Query:

```
?- test_all.
```

Expected:

```
d1 -> predicted: no, actual: no  
d2 -> predicted: no, actual: no  
d3 -> predicted: yes, actual: yes  
...  
d14 -> predicted: no, actual: no  
true.
```

Query:

```
?- accuracy.
```

Expected:

Accuracy: 14/14 (100.00%)
true.

Common Prolog Commands

Task	Command
Consult/load a file	?- [foil_prolog].
Consult file using path	?- ['foil/foil_prolog.pl'].
Run a query	?- classify(d9, Result).
Ask for another answer	Press ;
Stop asking for more answers	Press Enter
Interrupt a long query	Press Ctrl+C, then type a
Exit Prolog	?- halt.

Prolog Code Line-by-Line Explanation

Line:

```
:- ensure_loaded('..dataset/facts.pl').
```

Explanation:

This loads the dataset facts from facts.pl.

Line:

```
foil_rule(Day) :-  
    outlook(Day, overcast).
```

Explanation:

This is a rule. It says foil_rule(Day) is true if the outlook of that day is overcast.

Line:

```
foil_rule(Day) :-  
    outlook(Day, rainy),  
    wind(Day, weak).
```

Explanation:

This says a day is positive if the outlook is rainy and the wind is weak. The comma means AND.

Line:

```
classify(Day, yes) :-  
    foil_rule(Day),  
    !.
```

Explanation:

If any FOIL rule works for the day, classify it as yes. The ! is called cut. It tells Prolog not to search for another classification after finding yes.

Line:

```
classify(_, no).
```

Explanation:

The underscore _ means “anything”. If no yes rule worked, classify the day as no.

Line:

```
test_all :-  
    forall(day(Day), (...)).
```

Explanation:

This runs the classification for every day in the dataset.

Line:

```
accuracy :-  
    count_correct(Correct),  
    count_total(Total),  
    Percent is Correct * 100 / Total.
```

Explanation:

This calculates the accuracy percentage.

6. Project Folder Structure

```
foil-beam-assignment/  
|-- dataset/  
|   |-- data.csv  
|   `-- facts.pl  
|-- foil/  
|   |-- foil_python.py  
|   `-- foil_prolog.pl  
|-- beam_search/  
|   `-- beam_search.py  
|-- outputs/  
|   |-- foil_python_output.txt
```

```
| |-- foil_prolog_output.txt
| |-- beam_search_output.txt
|-- documentation/
    |-- assignment_report.md
```

7. Full Code Listings

dataset/facts.pl

% Play Tennis dataset written as Prolog facts.
% Each day has weather attributes and one target class.

```
day(d1).
outlook(d1, sunny).
temperature(d1, hot).
humidity(d1, high).
wind(d1, weak).
play_tennis(d1, no).
```

```
day(d2).
outlook(d2, sunny).
temperature(d2, hot).
humidity(d2, high).
wind(d2, strong).
play_tennis(d2, no).
```

```
day(d3).
outlook(d3, overcast).
temperature(d3, hot).
humidity(d3, high).
wind(d3, weak).
play_tennis(d3, yes).
```

```
day(d4).
outlook(d4, rainy).
temperature(d4, mild).
humidity(d4, high).
wind(d4, weak).
play_tennis(d4, yes).
```

```
day(d5).
outlook(d5, rainy).
temperature(d5, cool).
humidity(d5, normal).
wind(d5, weak).
```

play_tennis(d5, yes).

day(d6).

outlook(d6, rainy).

temperature(d6, cool).

humidity(d6, normal).

wind(d6, strong).

play_tennis(d6, no).

day(d7).

outlook(d7, overcast).

temperature(d7, cool).

humidity(d7, normal).

wind(d7, strong).

play_tennis(d7, yes).

day(d8).

outlook(d8, sunny).

temperature(d8, mild).

humidity(d8, high).

wind(d8, weak).

play_tennis(d8, no).

day(d9).

outlook(d9, sunny).

temperature(d9, cool).

humidity(d9, normal).

wind(d9, weak).

play_tennis(d9, yes).

day(d10).

outlook(d10, rainy).

temperature(d10, mild).

humidity(d10, normal).

wind(d10, weak).

play_tennis(d10, yes).

day(d11).

outlook(d11, sunny).

temperature(d11, mild).

humidity(d11, normal).

wind(d11, strong).

play_tennis(d11, yes).

```
day(d12).
outlook(d12, overcast).
temperature(d12, mild).
humidity(d12, high).
wind(d12, strong).
play_tennis(d12, yes).
```

```
day(d13).
outlook(d13, overcast).
temperature(d13, hot).
humidity(d13, normal).
wind(d13, weak).
play_tennis(d13, yes).
```

```
day(d14).
outlook(d14, rainy).
temperature(d14, mild).
humidity(d14, high).
wind(d14, strong).
play_tennis(d14, no).
```

```
positive_example(Day) :-
    play_tennis(Day, yes).
```

```
negative_example(Day) :-
    play_tennis(Day, no).
```

foil/foil_python.py

```
"""
```

Simplified FOIL-style rule learner for the Play Tennis dataset.

The program reads dataset/data.csv, separates positive and negative examples, then learns human-readable IF-THEN rules for play_tennis = yes.

```
"""
```

```
from __future__ import annotations
```

```
import csv
```

```
import math
```

```
from itertools import combinations
```

```
from pathlib import Path
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[1]
DATA_PATH = PROJECT_ROOT / "dataset" / "data.csv"
TARGET_COLUMN = "play_tennis"
POSITIVE_VALUE = "yes"
ATTRIBUTES = ["outlook", "temperature", "humidity", "wind"]
```

```
Condition = tuple[str, str]
Rule = tuple[Condition, ...]
```

```
def load_dataset(path: Path) -> list[dict[str, str]]:
    """Load the CSV file as a list of dictionaries."""
    with path.open(newline="", encoding="utf-8") as file:
        return list(csv.DictReader(file))
```

```
def get_domains(rows: list[dict[str, str]]) -> dict[str, list[str]]:
    """Collect possible values for each attribute."""
    domains: dict[str, list[str]] = {}
    for attribute in ATTRIBUTES:
        domains[attribute] = sorted({row[attribute] for row in rows})
    return domains
```

```
def covers(row: dict[str, str], rule: Rule) -> bool:
    """Return True if a row satisfies every condition in a rule."""
    return all(row[attribute] == value for attribute, value in rule)
```

```
def condition_text(rule: Rule) -> str:
    """Convert a rule condition tuple into readable text."""
    return " AND ".join(f"{attribute} = {value}" for attribute, value in rule)
```

```
def all_candidate_rules(domains: dict[str, list[str]], max_conditions: int = 2) -> list[Rule]:
    """Generate candidate rules with no repeated attribute."""
    literals = [
        (attribute, value)
        for attribute in ATTRIBUTES
        for value in domains[attribute]
    ]
```

```
    candidates: list[Rule] = []
```

```

for size in range(1, max_conditions + 1):
    for combo in combinations(literals, size):
        used_attributes = [attribute for attribute, _ in combo]
        if len(set(used_attributes)) != len(used_attributes):
            continue
        ordered_combo = tuple(
            sorted(combo, key=lambda item: ATTRIBUTES.index(item[0]))
        )
        candidates.append(ordered_combo)
return sorted(set(candidates), key=condition_text)

```

```

def foil_gain(
    remaining_positive_count: int,
    negative_count: int,
    covered_positive_count: int,
    covered_negative_count: int,
) -> float:
    """
    Compute a simple FOIL information gain score.

    Higher score means the rule keeps useful positive examples while removing
    negative examples.
    """
    if covered_positive_count == 0:
        return float("-inf")

    before_precision = remaining_positive_count / (
        remaining_positive_count + negative_count
    )
    after_precision = covered_positive_count / (
        covered_positive_count + covered_negative_count
    )
    return covered_positive_count * (
        math.log2(after_precision) - math.log2(before_precision)
    )

```

```

def learn_rules(rows: list[dict[str, str]]) -> list[Rule]:
    """Learn a small set of rules that cover positive examples."""
    domains = get_domains(rows)
    candidates = all_candidate_rules(domains, max_conditions=2)
    positives = [row for row in rows if row[TARGET_COLUMN] == POSITIVE_VALUE]

```

```

negatives = [row for row in rows if row[TARGET_COLUMN] != POSITIVE_VALUE]
uncovered_positive_ids = {row["id"] for row in positives}
learned_rules: list[Rule] = []

```

```

print("FOIL-like Rule Learning")
print("=====")
print(f"Dataset: {DATA_PATH}")
print(f"Positive examples: {', '.join(row['id'] for row in positives)}")
print(f"Negative examples: {', '.join(row['id'] for row in negatives)}")
print()

```

```

iteration = 1
while uncovered_positive_ids:
    print(f"Iteration {iteration}")
    print(
        "Uncovered positive examples: "
        + ", ".join(sorted(uncovered_positive_ids))
    )

```

```

scored_candidates = []
for rule in candidates:
    covered_remaining_positives = [
        row
        for row in positives
        if row["id"] in uncovered_positive_ids and covers(row, rule)
    ]
    covered_negatives = [row for row in negatives if covers(row, rule)]

```

```

if not covered_remaining_positives:
    continue

```

```

gain = foil_gain(
    remaining_positive_count=len(uncovered_positive_ids),
    negative_count=len(negatives),
    covered_positive_count=len(covered_remaining_positives),
    covered_negative_count=len(covered_negatives),
)
scored_candidates.append(
    {
        "rule": rule,
        "text": condition_text(rule),
        "positive_ids": [row["id"] for row in covered_remaining_positives],
        "negative_ids": [row["id"] for row in covered_negatives],
    }
)

```

```

        "positive_count": len(covered_remaining_positives),
        "negative_count": len(covered_negatives),
        "is_pure": len(covered_negatives) == 0,
        "length": len(rule),
        "gain": gain,
    }
)

scored_candidates.sort(
    key=lambda item: (
        item["is_pure"],
        item["positive_count"],
        -item["length"],
        item["gain"],
        item["text"],
    ),
    reverse=True,
)

print("Top candidate rules:")
for item in scored_candidates[:5]:
    print(
        f" IF {item['text']} THEN yes | "
        f"pos={item['positive_count']} {item['positive_ids']}, "
        f"neg={item['negative_count']} {item['negative_ids']}, "
        f"gain={item['gain']:.3f}"
    )

best = scored_candidates[0]
learned_rules.append(best["rule"])
for row_id in best["positive_ids"]:
    uncovered_positive_ids.remove(row_id)

print(f"Selected rule: IF {best['text']} THEN play_tennis = yes")
print()
iteration += 1

return learned_rules

def predict(row: dict[str, str], rules: list[Rule]) -> str:
    """Predict yes if any learned rule covers the row."""
    for rule in rules:

```

```

    if covers(row, rule):
        return "yes"
    return "no"

```

```

def print_classification(rows: list[dict[str, str]], rules: list[Rule]) -> None:
    """Print learned rules and predictions for all rows."""
    print("Final Learned Rules")
    print("=====")
    for index, rule in enumerate(rules, start=1):
        print(f"Rule {index}: IF {condition_text(rule)} THEN play_tennis = yes")
    print("Default rule: otherwise play_tennis = no")
    print()

    print("Classification Output")
    print("=====")
    correct = 0
    for row in rows:
        prediction = predict(row, rules)
        actual = row[TARGET_COLUMN]
        if prediction == actual:
            correct += 1
        print(
            f"{row['id']}: predicted={prediction}, actual={actual}, "
            f"features=({row['outlook']}, {row['temperature']}, "
            f"{row['humidity']}, {row['wind']})"
        )

    accuracy = correct / len(rows) * 100
    print()
    print(f"Accuracy: {correct}/{len(rows)} = {accuracy:.2f}%")

```

```

def main() -> None:
    rows = load_dataset(DATA_PATH)
    rules = learn_rules(rows)
    print_classification(rows, rules)

```

```

if __name__ == "__main__":
    main()

```

foil/foil_prolog.pl

```
% FOIL-style learned rules for the Play Tennis dataset.  
% Load this file in SWI-Prolog from the foil folder using:  
% ?- [foil_prolog].
```

```
:- ensure_loaded('..dataset/facts.pl').
```

```
% Rule 1:
```

```
% If outlook is overcast, tennis is played.
```

```
foil_rule(Day) :-  
    outlook(Day, overcast).
```

```
% Rule 2:
```

```
% If outlook is rainy and wind is weak, tennis is played.
```

```
foil_rule(Day) :-  
    outlook(Day, rainy),  
    wind(Day, weak).
```

```
% Rule 3:
```

```
% If outlook is sunny and humidity is normal, tennis is played.
```

```
foil_rule(Day) :-  
    outlook(Day, sunny),  
    humidity(Day, normal).
```

```
% Classification rule.
```

```
% If any FOIL rule succeeds, predict yes. Otherwise predict no.
```

```
classify(Day, yes) :-  
    foil_rule(Day),  
    !.  
classify(_, no).
```

```
% Print a readable explanation for one day.
```

```
explain(Day) :-  
    outlook(Day, Outlook),  
    temperature(Day, Temperature),  
    humidity(Day, Humidity),  
    wind(Day, Wind),  
    classify(Day, Prediction),  
    play_tennis(Day, Actual),  
    format('~w: outlook=~w, temperature=~w, humidity=~w, wind=~w~n',  
        [Day, Outlook, Temperature, Humidity, Wind]),  
    format('Predicted: ~w, Actual: ~w~n', [Prediction, Actual]).
```

% Test every row in the dataset.

```
test_all :-  
  forall(  
    day(Day),  
    (  
      classify(Day, Prediction),  
      play_tennis(Day, Actual),  
      format('~w -> predicted: ~w, actual: ~w~n',  
        [Day, Prediction, Actual])  
    )  
  ).
```

% Count correct predictions.

```
count_correct(Count) :-  
  findall(  
    Day,  
    (  
      day(Day),  
      classify(Day, Prediction),  
      play_tennis(Day, Prediction)  
    ),  
    CorrectDays  
  ),  
  length(CorrectDays, Count).
```

% Count all dataset rows.

```
count_total(Count) :-  
  findall(Day, day(Day), Days),  
  length(Days, Count).
```

% Print accuracy.

```
accuracy :-  
  count_correct(Correct),  
  count_total(Total),  
  Percent is Correct * 100 / Total,  
  format('Accuracy: ~w/~w (~2f%)~n', [Correct, Total, Percent]).
```

beam_search/beam_search.py

"""

Beam Search demonstration using the Play Tennis dataset.

The search space contains possible IF-rules such as:

IF outlook = rainy AND wind = weak THEN play_tennis = yes

Beam Search does not keep every candidate. At each level it keeps only the best few candidates according to a scoring function.

"""

```
from __future__ import annotations
```

```
import csv
```

```
from pathlib import Path
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[1]
```

```
DATA_PATH = PROJECT_ROOT / "dataset" / "data.csv"
```

```
TARGET_COLUMN = "play_tennis"
```

```
POSITIVE_VALUE = "yes"
```

```
ATTRIBUTES = ["outlook", "temperature", "humidity", "wind"]
```

```
BEAM_WIDTH = 3
```

```
MAX_DEPTH = 2
```

```
Condition = tuple[str, str]
```

```
State = tuple[Condition, ...]
```

```
def load_dataset(path: Path) -> list[dict[str, str]]:
```

```
    with path.open(newline="", encoding="utf-8") as file:
```

```
        return list(csv.DictReader(file))
```

```
def get_domains(rows: list[dict[str, str]]) -> dict[str, list[str]]:
```

```
    return {
```

```
        attribute: sorted({row[attribute] for row in rows})
```

```
        for attribute in ATTRIBUTES
```

```
    }
```

```
def normalize_state(state: State) -> State:
```

```
    return tuple(sorted(state, key=lambda item: ATTRIBUTES.index(item[0])))
```

```
def state_text(state: State) -> str:
```

```
    if not state:
```

```
        return "TRUE"
```

```
    return " AND ".join(f"{attribute} = {value}" for attribute, value in state)
```

```

def covers(row: dict[str, str], state: State) -> bool:
    return all(row[attribute] == value for attribute, value in state)

def evaluate_state(rows: list[dict[str, str]], state: State) -> dict[str, float | int]:
    covered = [row for row in rows if covers(row, state)]
    positive = [row for row in covered if row[TARGET_COLUMN] == POSITIVE_VALUE]
    negative = [row for row in covered if row[TARGET_COLUMN] != POSITIVE_VALUE]
    total_positive = sum(1 for row in rows if row[TARGET_COLUMN] == POSITIVE_VALUE)

    precision = len(positive) / len(covered) if covered else 0.0
    recall = len(positive) / total_positive if total_positive else 0.0

    # This score prefers accurate rules, but still rewards rules that cover
    # several positive examples. Pure rules get a small bonus.
    score = (0.70 * precision) + (0.30 * recall)
    if negative == [] and positive:
        score += 0.05
    score -= 0.02 * max(0, len(state) - 1)

    return {
        "positive": len(positive),
        "negative": len(negative),
        "covered": len(covered),
        "precision": precision,
        "recall": recall,
        "score": score,
    }

def expand_state(state: State, domains: dict[str, list[str]]) -> list[State]:
    used_attributes = {attribute for attribute, _ in state}
    children: list[State] = []

    for attribute in ATTRIBUTES:
        if attribute in used_attributes:
            continue
        for value in domains[attribute]:
            child = normalize_state(state + ((attribute, value),))
            children.append(child)

    return children

```

```

def rank_states(rows: list[dict[str, str]], states: list[State]) -> list[tuple[State, dict[str, float | int]]]:
    unique_states = sorted(set(states), key=state_text)
    scored = [(state, evaluate_state(rows, state)) for state in unique_states]
    scored.sort(
        key=lambda item: (
            item[1]["score"],
            item[1]["positive"],
            -len(item[0]),
            state_text(item[0]),
        ),
        reverse=True,
    )
    return scored

```

```

def print_state(prefix: str, state: State, stats: dict[str, float | int]) -> None:
    print(
        f'{prefix}IF {state_text(state)} THEN play_tennis = yes | '
        f'pos={stats['positive']}, neg={stats['negative']}, '
        f'precision={stats['precision']:.2f}, recall={stats['recall']:.2f}, '
        f'score={stats['score']:.3f}'
    )

```

```

def beam_search(rows: list[dict[str, str]]) -> State:
    domains = get_domains(rows)
    beam: list[State] = [tuple()]
    best_state: State = tuple()
    best_stats = evaluate_state(rows, best_state)

    print("Beam Search for a Good Play-Tennis Rule")
    print("=====")
    print(f"Dataset: {DATA_PATH}")
    print(f"Initial state: IF TRUE THEN play_tennis = yes")
    print(f"Beam width: {BEAM_WIDTH}")
    print(f"Maximum depth: {MAX_DEPTH}")
    print("Scoring function: 0.70 * precision + 0.30 * recall + purity bonus")
    print()

    for depth in range(1, MAX_DEPTH + 1):

```

```

print(f"Level {depth}: expand current beam")
candidates: list[State] = []
for state in beam:
    candidates.extend(expand_state(state, domains))

ranked = rank_states(rows, candidates)
print("Candidate states, sorted by score:")
for state, stats in ranked[:8]:
    print_state(" ", state, stats)

beam = [state for state, _ in ranked[:BEAM_WIDTH]]
print("Selected states kept in the beam:")
for state in beam:
    stats = evaluate_state(rows, state)
    print_state(" ", state, stats)
    if (
        stats["score"] > best_stats["score"]
        or (
            stats["score"] == best_stats["score"]
            and len(state) < len(best_state)
        )
    ):
        best_state = state
        best_stats = stats
print()

print("Final Result")
print("=====")
print_state("Best rule found: ", best_state, best_stats)
print("Meaning: if the condition is true, predict play_tennis = yes.")
print("Otherwise, use the default prediction play_tennis = no.")
return best_state

def main() -> None:
    rows = load_dataset(DATA_PATH)
    beam_search(rows)

if __name__ == "__main__":
    main()

```

8. Commands to Run

From inside:

```
foil-beam-assignment
```

Run FOIL Python:

```
python foil/foil_python.py
```

Save FOIL Python output:

```
python foil/foil_python.py > outputs/foil_python_output.txt
```

Run Beam Search:

```
python beam_search/beam_search.py
```

Save Beam Search output:

```
python beam_search/beam_search.py > outputs/beam_search_output.txt
```

Run Prolog from the project folder:

```
?- ['foil/foil_prolog.pl'].
```

```
?- classify(d9, Result).
```

```
?- test_all.
```

```
?- accuracy.
```

```
?- halt.
```

Run Prolog from the foil folder:

```
?- [foil_prolog].
```

```
?- classify(d9, Result).
```

```
?- test_all.
```

```
?- accuracy.
```

```
?- halt.
```

9. Result Discussion

The FOIL-style rules classify all 14 examples correctly.

The final rule set is understandable:

```
IF outlook = overcast THEN yes
```

```
IF outlook = rainy AND wind = weak THEN yes
```

```
IF outlook = sunny AND humidity = normal THEN yes
```

```
OTHERWISE no
```

The Beam Search program demonstrates how the algorithm keeps only the best few candidate rules at each level. With beam width 3, it does not search every possible rule deeply. Instead, it uses the score to focus on promising rules.

10. Conclusion

This project shows how FOIL and Beam Search can be applied to the same dataset.

FOIL is useful for learning logical IF-THEN rules from positive and negative examples. Prolog is suitable for representing these rules because Prolog is based on facts, rules, and queries.

Beam Search is useful for searching through candidate rules efficiently by keeping only the best few candidates at each level.

Both implementations produce readable output, making them suitable for assignment demonstration and screenshot collection.